



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE CRATEÚS
SEMESTRE 2026.1

FRANCISCO DAGOBERTO SILVA DOS SANTOS – 554237
MARIA FERNANDA FERREIRA PAULINO - 552554
LUIZ MATHEUS SALES SOUZA – 586937

PROBLEMA DO CAIXEIRO VIAJANTE
ALGORÍTMOS EM GRAFOS
TRABALHO PRÁTICO

CRATEÚS
2026

SUMÁRIO

1	INTRODUÇÃO	2
1.1	DESCRIÇÃO DO PROBLEMA E MOTIVAÇÃO	2
1.2	OBJETIVOS DO TRABALHO	3
2	MÉTODO PROPOSTO	4
2.1	MÉTODO IMPLEMENTADO	4
2.2	PSEUDOCÓDIGO	7
2.3	ANÁLISE CONCEITUAL	10
2.4	JUSTIFICATIVA DAS DECISÕES	11
3	RESULTADOS E DISCUSSÕES	13
3.1	INSTÂNCIAS UTILIZADAS	13
3.2	METODOLOGIA DE EXPERIMENTAÇÃO	14
3.3	RESULTADOS OBTIDOS	14
3.4	PONTOS FORTES	24
3.5	LIMITAÇÕES	24
3.6	POSSÍVEIS MELHORIAS	25
4	CONCLUSÃO	26
5	USO DE FERRAMENTAS DE IA	27
5.1	FERRAMENTAS UTILIZADAS	27
5.2	FINALIDADES DE USO	27
5.3	PROMPTS UTILIZADOS	28
	REFERÊNCIAS	29

1 INTRODUÇÃO

Este relatório tem como objetivo apresentar e descrever, de forma detalhada, o segundo trabalho prático da disciplina de Algoritmos em Grafos. Para isso, são abordados os principais conceitos teóricos e os algoritmos empregados no desenvolvimento da solução computacional para o Problema do Caixeiro Viajante (PCV), destacando as técnicas utilizadas, as decisões de implementação adotadas e os resultados obtidos.

Além desta introdução, o relatório está organizado da seguinte forma. Inicialmente, apresenta-se a motivação do trabalho, destacando a relevância do PCV e suas principais aplicações, bem como os objetivos que orientaram o desenvolvimento da atividade. Em seguida, é descrito o método proposto, contemplando a apresentação detalhada do algoritmo implementado, seu pseudocódigo, a análise conceitual dos fundamentos teóricos envolvidos e as justificativas para as decisões adotadas durante o desenvolvimento da solução.

Posteriormente, são apresentados os resultados e discussões, incluindo a descrição das instâncias utilizadas, a metodologia de experimentação empregada, os resultados obtidos e uma análise crítica do desempenho da solução. Nessa etapa, também são discutidos os principais pontos fortes, as limitações observadas e possíveis melhorias que podem ser exploradas em trabalhos futuros. Por fim, são apresentadas as conclusões do trabalho, destacando os principais resultados alcançados e os conhecimentos adquiridos ao longo do desenvolvimento da atividade.

1.1 DESCRIÇÃO DO PROBLEMA E MOTIVAÇÃO

A definição formal do **Problema do Caixeiro Viajante** (do inglês, *Traveling Salesman Problem – TSP*) é dada como segue:

“Dado um grafo $G = (V, E)$ e uma função de custo nas arestas $c : E \rightarrow \mathbb{R}^+$, determinar um ciclo que visite cada vértice (exatamente uma vez) tal que a soma dos pesos das arestas que o compõem seja a menor possível.”

Em outras palavras, o TSP consiste em determinar um ciclo hamiltoniano de custo mínimo em um grafo ponderado, direcionado ou não. Trata-se de um dos problemas mais clássicos e estudados nas áreas de Teoria dos Grafos, Pesquisa Operacional, Otimização Combinatória e Ciência da Computação, servindo como referência para o desenvolvimento e avaliação de técnicas de otimização e algoritmos (WANG *et al.*, 2023).

A relevância do problema está associada tanto à sua importância teórica quanto às suas diversas aplicações práticas. Problemas relacionados ao planejamento de rotas de entrega, transporte de mercadorias (SACRAMENTO *et al.*, 2019), logística empresarial, roteamento de veículos, coleta de dados em redes de sensores e sequenciamento de operações industriais podem ser modelados como instâncias do PCV. Nesses cenários, a determinação de rotas eficientes pode contribuir para a redução de custos operacionais, melhor utilização de recursos e aumento da eficiência dos processos.

Além de sua ampla aplicabilidade, o TSP apresenta elevado grau de dificuldade computacional. Mesmo em versões simplificadas do problema, como aquelas em que os pesos das arestas pertencem ao conjunto $\{1,2\}$, encontrar uma solução ótima permanece uma tarefa complexa. De fato, determinar um ciclo hamiltoniano de custo mínimo em um grafo arbitrário é um problema NP-difícil, não sendo conhecidos algoritmos capazes de resolvê-lo em tempo polinomial para o caso geral (WANG *et al.*, 2023). À medida que o número de vértices aumenta, a quantidade de possíveis rotas cresce rapidamente, tornando a busca pela melhor solução um desafio significativo.

Nesse contexto, o estudo do PCV permite a aplicação de conceitos fundamentais da disciplina de Algoritmos em Grafos, tais como ciclos hamiltonianos, otimização combinatória e análise de complexidade computacional. Além disso, a implementação e avaliação de algoritmos voltados à sua resolução possibilitam compreender os desafios envolvidos no desenvolvimento de soluções eficientes para problemas computacionalmente complexos.

1.2 OBJETIVOS DO TRABALHO

Este trabalho tem como objetivo implementar e avaliar algoritmos para a resolução do PCV, utilizando instâncias geométricas descritas no formato da Travelling Salesman Problem Library (TSPLIB). Além da implementação das soluções propostas, busca-se analisar seu desempenho e a qualidade dos resultados obtidos, por meio da realização de experimentos computacionais. Os resultados serão apresentados e discutidos em sala de aula, permitindo a avaliação das abordagens adotadas e a comparação de seu comportamento em diferentes instâncias do problema.

2 MÉTODO PROPOSTO

Para a resolução do problema proposto, foi adotado o algoritmo de Colônia de Formigas (*Ant Colony Optimization – ACO*). O funcionamento do algoritmo e as decisões de implementação adotadas neste trabalho serão detalhados nas seções seguintes.

2.1 MÉTODO IMPLEMENTADO

O algoritmo ACO, implementado neste trabalho, é uma metaheurística inspirada no comportamento coletivo observado em colônias de formigas reais. Na natureza, as formigas exploram o ambiente em busca de alimento e, ao encontrarem uma fonte de comida, depositam feromônios ao longo do caminho percorrido. Com o tempo, outras formigas tendem a seguir as trilhas com maior concentração de feromônio, reforçando-as ainda mais. Esse mecanismo permite que a colônia encontre rotas eficientes entre o formigueiro e a fonte de alimento (AKHTAR, 2019).

A Figura 1 ilustra o processo de convergência do algoritmo ACO. As formigas exploram diferentes rotas e reforçam, por meio da deposição de feromônio, o caminho de menor distância. Com o tempo, a maioria das formigas passa a utilizar essa rota, caracterizando a identificação da solução ótima.

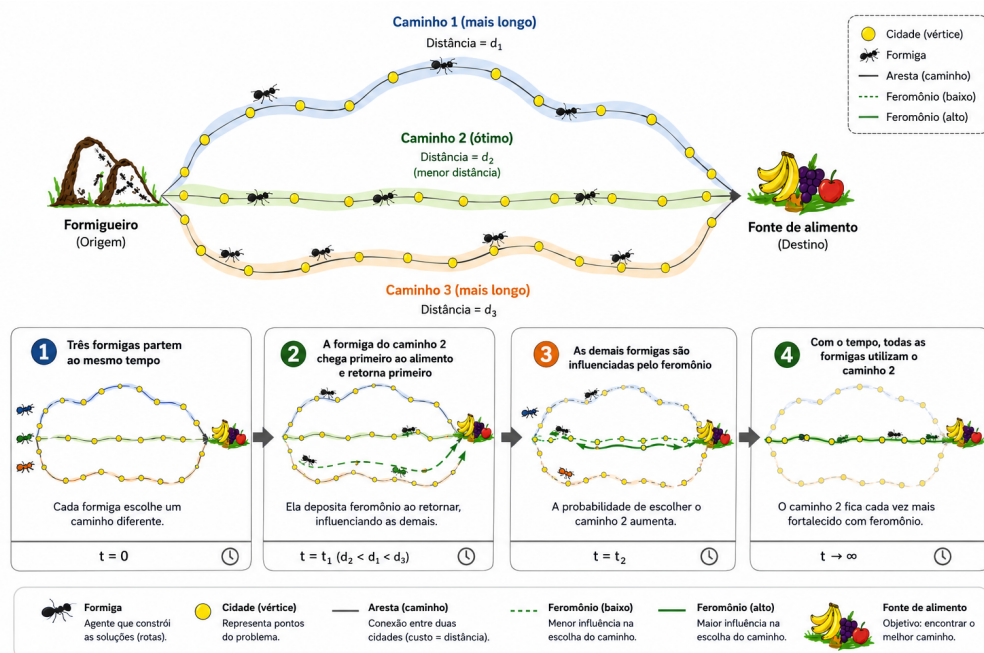


Figura 1 – Representação do processo de convergência do algoritmo ACO por meio do reforço de feromônios em caminhos de diferentes comprimentos. Fonte: Imagem Adaptada pelos autores a partir de material educacional em vídeo.(Ponto Ótimo, 2017)

No contexto do PCV, cada formiga representa uma solução candidata. Inicialmente, uma formiga parte de uma cidade e constrói uma rota visitando sucessivamente cidades ainda não visitadas até completar um ciclo hamiltoniano. A escolha da próxima cidade é realizada de forma probabilística, considerando tanto a quantidade de feromônio presente nas arestas quanto uma informação heurística relacionada à distância entre as cidades.

A probabilidade de uma formiga localizada na cidade (i) escolher a próxima cidade (j) é calculada pela Equação 2.1:

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \eta_{ij}^{\beta}}{\sum_{k \in N_i} \tau_{ik}^{\alpha} \eta_{ik}^{\beta}} \quad (2.1)$$

onde:

P_{ij} Probabilidade de a formiga deslocar-se da cidade i para a cidade j .

τ_{ij} Quantidade de feromônio presente na aresta que conecta as cidades i e j .

η_{ij} Informação heurística utilizada para orientar a busca da próxima cidade.

α Parâmetro que controla a influência do feromônio na tomada de decisão.

β Parâmetro que controla a influência da informação heurística na tomada de decisão.

A informação heurística é definida como o inverso da distância entre duas cidades, conforme apresentado na Equação 2.2:

$$\eta_{ij} = \frac{1}{d_{ij}} \quad (2.2)$$

em que d_{ij} representa a distância entre as cidades i e j .

Observa-se que a probabilidade de escolha aumenta quando uma aresta apresenta maior concentração de feromônio ou quando conecta cidades mais próximas. Dessa forma, o algoritmo busca equilibrar o aproveitamento do conhecimento acumulado durante a execução e a exploração de novas possibilidades de solução.

Durante o desenvolvimento da solução, foi implementado um mecanismo denominado radar, cujo objetivo é reduzir o custo computacional da construção das rotas. O radar, limita a quantidade de cidades avaliadas em cada decisão. Em vez de analisar todas as cidades ainda não visitadas, a formiga considera apenas um subconjunto previamente definido. Por exemplo, em

uma instância com 100 cidades e um radar configurado para 20%, apenas 20 cidades candidatas são analisadas em cada etapa da construção da rota.

Após a construção das rotas, ocorre o processo de evaporação dos feromônios, responsável por evitar que o algoritmo fique excessivamente concentrado em soluções exploradas anteriormente. Esse mecanismo reduz gradualmente a influência de trilhas antigas e favorece a exploração de novas regiões do espaço de busca. A evaporação é realizada de acordo com a Equação 2.3:

$$\tau_{ij} = (1 - \rho) \tau_{ij} \quad (2.3)$$

onde ρ representa a taxa de evaporação dos feromônios, assumindo valores no intervalo $0 \leq \rho \leq 1$.

Após a evaporação, é realizada a etapa de reforço dos feromônios, na qual as soluções de melhor qualidade recebem maior quantidade de feromônio. A atualização é descrita pela Equação 2.4:

$$\tau_{ij} = \tau_{ij} + \Delta \tau_{ij} \quad (2.4)$$

em que a quantidade de feromônio adicionada é calculada pela Equação 2.5:

$$\Delta \tau_{ij} = \frac{Q}{L} \quad (2.5)$$

onde:

- (Q) é uma constante associada à intensidade do reforço de feromônio;
- (L) corresponde ao custo total da rota construída pela formiga.

Dessa forma, rotas com menor custo recebem maior quantidade de feromônio, aumentando a probabilidade de que caminhos promissores sejam explorados nas iterações subsequentes. Esse mecanismo permite que o algoritmo compartilhe informações entre as formigas e direcione gradualmente a busca para regiões mais promissoras do espaço de soluções.

O processo completo é repetido ao longo de diversas iterações até que um critério de parada seja atingido, como um número máximo de iterações, um limite de tempo de execução ou a convergência da melhor solução encontrada.

2.2 PSEUDOCÓDIGO

Algoritmo 1: Geração das Cidades do Problema TSP

Input: Número de cidades n_{cidades}

Output: Conjunto de coordenadas das cidades

Inicializar matriz $coords$;

for $i \leftarrow 1$ até n_{cidades} **do**

 Gerar coordenada aleatória $x \in [0, 1000]$;

 Gerar coordenada aleatória $y \in [0, 1000]$;

$coords[i] \leftarrow (x, y)$;

end

Criar estrutura $dados_{\text{teste}}$;

$dados_{\text{teste}}[\"DIMENSION\"] \leftarrow n_{\text{cidades}}$;

$dados_{\text{teste}}[\"COORDS\"] \leftarrow coords$;

Criar ambiente $env \leftarrow TSPEnvironment(dados_{\text{teste}})$;

return env ;

Algoritmo 2: Instanciação da Colônia

Input: Número de formigas n_{formigas} , parâmetros α e β

Output: Lista de formigas

Inicializar lista $formigas \leftarrow \emptyset$;

for $i \leftarrow 1$ até n_{formigas} **do**

 Criar formiga $f_i \leftarrow Ant(\alpha, \beta)$;

 Adicionar f_i em $formigas$;

end

return $formigas$;

Algoritmo 3: Percurso de uma Formiga

Input: Ambiente env , matriz de feromônio F

Output: Rota construída e distância total

Inicializar rota com cidade inicial 0;

Inicializar conjunto $naoVisitadas$;

$distanciaTotal \leftarrow 0$;

$atual \leftarrow 0$;

while $naoVisitadas \neq \emptyset$ **do**

 Escolher próxima cidade utilizando:

$$P_{ij} = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum (\tau_{ik})^\alpha (\eta_{ik})^\beta}$$

$proxima \leftarrow escolher_proxima()$;

$distanciaTotal \leftarrow distanciaTotal + env.distancias[atual][proxima]$;

 Adicionar $proxima$ na rota;

 Remover $proxima$ de $naoVisitadas$;

$atual \leftarrow proxima$;

end

Adicionar retorno para cidade inicial;

Atualizar $distanciaTotal$;

return rota, $distanciaTotal$;

Algoritmo 4: Atualização dos Feromônios

Input: Lista de formigas, matriz de feromônio F , taxa de evaporação ρ , constante Q

Output: Nova matriz de feromônio

```

foreach aresta  $(i, j)$  do
     $F_{ij} \leftarrow (1 - \rho) \cdot F_{ij};$ 
end

foreach formiga  $k$  do
     $\Delta\tau \leftarrow \frac{Q}{distanciaTotal_k};$ 
    foreach aresta  $(i, j)$  da rota da formiga do
         $F_{ij} \leftarrow F_{ij} + \Delta\tau;$ 
         $F_{ji} \leftarrow F_{ji} + \Delta\tau;$ 
    end
end

return  $F$ ;
  
```

Algoritmo 5: Armazenamento do Histórico

Input: Iteração atual i , melhor distância global, matriz de feromônio

Output: Histórico atualizado

Adicionar melhor distância global em *historicoDistancias*;

historicoFeromonios $[i] \leftarrow copia(matrizFeromonio);$

return histórico atualizado;

Algoritmo 6: Fluxo Geral da Colônia de Formigas

Input: Número de cidades, número de formigas, número de iterações

Output: Melhor rota encontrada

Gerar cidades;

Calcular matriz de distâncias;

Inicializar matriz de feromônio;

for *iteracao* $\leftarrow$ 1 até *n_iteracoes* **do**

 Instanciar colônia de formigas;

foreach *formiga* **do**

 Construir rota completa;

 Calcular distância da rota;

 Atualizar melhor solução global;

end

 Evaporar feromônios;

 Depositar novos feromônios;

 Registrar histórico da iteração;

end

return melhor rota global;

2.3 ANÁLISE CONCEITUAL

O Problema do Caixeiro Viajante pode ser modelado como um grafo ponderado completo, em que cada vértice representa uma cidade e cada aresta representa o custo de deslocamento entre duas cidades. O objetivo consiste em encontrar um ciclo hamiltoniano de custo mínimo, isto é, uma rota que visite todas as cidades exatamente uma vez e retorne à cidade de origem. Como o Problema do Caixeiro Viajante pertence à classe dos problemas NP-difíceis, o número de soluções possíveis cresce exponencialmente com o aumento da quantidade de cidades. Dessa forma, métodos exatos tornam-se inviáveis para instâncias de grande porte, motivando a utilização de metaheurísticas capazes de encontrar soluções de boa qualidade em tempo computacional reduzido.

Nesse contexto, o algoritmo de Colônia de Formigas (ACO) utiliza uma estratégia inspirada no comportamento coletivo das formigas para realizar a busca por soluções. O processo combina dois mecanismos principais: a exploração do espaço de busca por meio da escolha

probabilística de caminhos e a intensificação da busca em regiões promissoras por meio do reforço dos feromônios. Esse equilíbrio entre exploração e intensificação é fundamental para evitar tanto a convergência prematura quanto a busca excessivamente aleatória. A decisão de cada formiga é influenciada por dois fatores: a quantidade de feromônio presente nas arestas e uma informação heurística baseada na distância entre as cidades. O feromônio representa o conhecimento acumulado ao longo das iterações, enquanto a heurística favorece a escolha de cidades mais próximas. Os parâmetros (α) e (β) controlam a importância relativa desses dois componentes, permitindo ajustar o comportamento do algoritmo de acordo com as características do problema.

Outro conceito importante utilizado na implementação foi a redução do espaço de busca por meio do mecanismo denominado radar. Essa estratégia restringe o conjunto de cidades candidatas avaliadas em cada decisão, diminuindo significativamente o custo computacional da construção das rotas. Sob o ponto de vista conceitual, essa abordagem representa uma troca entre eficiência e qualidade da solução, uma vez que reduz o tempo de execução ao custo de uma pequena diminuição da capacidade exploratória do algoritmo.

Por fim, os mecanismos de evaporação e deposição de feromônios desempenham papel fundamental no aprendizado coletivo das formigas artificiais. Enquanto a evaporação reduz a influência de informações antigas, evitando a estagnação da busca, a deposição reforça as arestas pertencentes às melhores soluções encontradas. Esse processo iterativo permite que o algoritmo converja gradualmente para regiões cada vez mais promissoras do espaço de soluções, aumentando a probabilidade de encontrar rotas de baixo custo para o Problema do Caixeiro Viajante.

2.4 JUSTIFICATIVA DAS DECISÕES

A concepção da solução baseou-se na otimalidade da solução e o tempo de execução, requisitos avaliados na resolução do Problema do Caixeiro Viajante. A metaheurística de Otimização por Colônia de Formigas foi adotada devido à sua capacidade de explorar eficientemente o espaço de busca em grafos, utilizando a retroalimentação de feromônios para guiar a convergência em tempo hábil.

Objetivando reduzir o custo computacional durante a construção das rotas, foi agregado ao algoritmo uma espécie de "radar", responsável por restringir o conjunto de candidatos avaliados em cada etapa da busca. Em vez de considerar todos os vértices ainda não visitados,

a seleção passa a ocorrer sobre um subconjunto previamente definido de vizinhos promissores. Essa estratégia busca reduzir a quantidade de operações realizadas a cada decisão das formigas, melhorando a eficiência para instâncias de maior porte. Essa escolha procura unir fatores relacionados ao desempenho computacional e a qualidade da solução, pois, a proposta limita o espaço de busca, impedindo a avaliação de algumas alternativas vantajosas. Além disso, a redução do esforço computacional permite a realização de um número maior de iterações dentro do tempo disponível, favorecendo o processo global de exploração.

O principal diferencial metodológico da abordagem implementada consiste na utilização de um modelo de Inteligência Artificial para estimar os valores dos hiperparâmetros α e β , responsáveis por controlar, respectivamente, a influência do feromônio e da informação heurística durante a tomada de decisão das formigas. Em implementações convencionais, esses parâmetros são normalmente definidos de forma empírica e permanecem constantes ao longo de toda a execução. Neste trabalho, optou-se por inferi-los dinamicamente a partir de características extraídas da instância analisada, permitindo que o algoritmo adapte seu comportamento às particularidades do problema resolvido.

Espera-se que essa estratégia reduza a dependência de inserções manuais e possibilite uma configuração inicial mais adequada para diferentes cenários de execução. Embora a etapa de inferência introduza um custo computacional adicional associado ao carregamento do modelo e das bibliotecas necessárias, tal custo é amortizado ao longo da execução do algoritmo. Em contrapartida, a abordagem oferece maior capacidade de adaptação frente a instâncias com características estruturais distintas, constituindo uma alternativa à utilização de conjuntos fixos de parâmetros previamente inseridos.

3 RESULTADOS E DISCUSSÕES

Os resultados obtidos indicam que a integração entre o algoritmo ACO e o modelo de Inteligência Artificial foi capaz de fornecer valores iniciais para os parâmetros α e β compatíveis com as características das instâncias avaliadas. Como consequência, observou-se a construção de soluções competitivas já nas primeiras iterações do algoritmo.

Entretanto, verificou-se que a utilização do modelo introduz um custo computacional adicional associado ao carregamento das bibliotecas e à etapa de inferência. Em ambientes sujeitos a restrições rígidas de tempo, esse custo reduz a quantidade de iterações que podem ser executadas pelo ACO durante a busca por soluções.

Observa-se que a definição inicial dos parâmetros afeta a capacidade de refinamento das soluções durante as iterações subsequentes. Nas instâncias de menor porte, a parametrização obtida pelo modelo mostrou-se suficiente para alcançar resultados satisfatórios. Por outro lado, nas instâncias de maior dimensão, a redução do número de iterações disponíveis limitou o refinamento das soluções, resultando em distâncias finais superiores às melhores soluções conhecidas para essas instâncias.

3.1 INSTÂNCIAS UTILIZADAS

As avaliações foram realizadas utilizando instâncias disponibilizadas pela *Travelling Salesman Problem Library* (TSPLIB). A utilização dessas instâncias permite a reprodução das avaliações e possibilita a comparação dos resultados obtidos com aqueles apresentados no documento do trabalho.

O conjunto de testes utiliza instâncias simétricas representadas por coordenadas euclidianas bidimensionais, abrangendo diferentes tamanhos. Entre elas encontram-se instâncias menores, como `tsp51.tsp`, composta por 51 vértices, e instâncias de maior dimensão, como `tsp436.tsp`, contendo 436 vértices. A variedade de tamanhos presente no conjunto fornecido possibilita avaliar o comportamento da abordagem proposta em problemas com diferentes níveis de complexidade computacional.

A utilização dessas instâncias permite analisar o impacto dos mecanismos incorporados ao algoritmo, especialmente a estratégia de restrição de vizinhança implementada por meio do radar e a inferência dinâmica dos hiperparâmetros α e β . Além disso, a diversidade do conjunto de testes possibilita observar como o tempo de execução e a qualidade das soluções se

comportam à medida que o número de vértices da instância aumenta.

3.2 METODOLOGIA DE EXPERIMENTAÇÃO

Os experimentos foram realizados em ambientes heterogêneos, nos sistemas operacionais Windows e em distribuições baseadas em Linux. Essa escolha visou observar o desempenho da implementação em diferentes ambientes. O algoritmo foi desenvolvido utilizando a linguagem Python, com as bibliotecas nativas `sys` e `math`, além da biblioteca `numpy` para manipulação de estruturas matriciais empregadas durante a execução da metaheurística.

A execução dos testes ocorreu por meio da interface de linha de comando, utilizando a entrada padrão para leitura das instâncias no formato `.tsp`. Os parâmetros de controle do algoritmo, incluindo os valores de α , β e o limite de iterações, foram definidos a partir das informações contidas no arquivo `parametros_gerais_ia.csv`, gerado previamente pelo modelo de IA utilizado neste trabalho.

Para reduzir a influência da aleatoriedade inerente ao ACO, foi adotada a utilização de sementes fixas nos geradores de números pseudoaleatórios. Além disso, os parâmetros inferidos foram mantidos constantes para cada instância durante a execução dos experimentos. As métricas analisadas compreenderam o custo da melhor rota encontrada e o tempo total de execução do algoritmo.

O principal critério de parada adotado foi o número máximo de iterações, definido em função do tamanho da instância analisada. Essa estratégia permitiu adequar o esforço computacional às diferentes dimensões dos problemas avaliados, possibilitando uma comparação mais consistente entre os resultados obtidos.

3.3 RESULTADOS OBTIDOS

O mecanismo denominado radar, introduzido na seção 2.1, contribuiu significativamente para a melhoria dos resultados obtidos. Na versão tradicional do ACO, para cada cidade visitada, a formiga avalia todas as cidades ainda não visitadas a fim de determinar a próxima escolha.

Considerando uma instância com n cidades, o número total de avaliações realizadas durante a construção de uma rota pode ser expresso pela Equação 3.1:

$$A(n) = (n-1) + (n-2) + (n-3) + \dots + 1 \quad (3.1)$$

onde $A(n)$ representa a quantidade total de avaliações necessárias para a construção de uma rota em uma instância com n cidades.

A Equação 3.1 corresponde à soma dos termos de uma progressão aritmética (PA), cuja fórmula é mostrada na Equação 3.2:

$$S_n = \frac{(a_1 + a_n)n}{2} \quad (3.2)$$

No caso da Equação 3.1, o número de termos é igual a $(n-1)$. Portanto:

$$\begin{aligned} a_1 &= n-1 \\ a_n &= 1 \\ N &= n-1 \end{aligned} \quad (3.3)$$

Substituindo esses valores na Equação 3.2, obtém-se:

$$A(n) = \frac{[(n-1) + 1](n-1)}{2} \quad (3.4)$$

Simplificando:

$$A(n) = \frac{n(n-1)}{2} \quad (3.5)$$

$$A(n) = \frac{n^2 - n}{2} \quad (3.6)$$

Observa-se que o termo dominante da expressão 3.6 é de ordem quadrática. Assim, a complexidade assintótica da etapa de construção de uma rota é dada por:

$$O(n^2) \quad (3.7)$$

Com a introdução do mecanismo radar, o comportamento do algoritmo é alterado. Em vez de considerar todas as cidades ainda não visitadas, a formiga passa a avaliar apenas

um subconjunto previamente definido de candidatos. Por exemplo, em uma instância com 100 cidades e um radar configurado para 20%, apenas 20 cidades são analisadas a cada etapa da construção da rota.

Como a quantidade de cidades avaliadas passa a ser constante, a complexidade da construção da solução torna-se aproximadamente linear em relação ao número de cidades, tendo sua notação assintótica 3.8:

$$O(n) \quad (3.8)$$

Embora essa estratégia reduza a capacidade exploratória do algoritmo, os experimentos realizados indicam que a perda de qualidade das soluções foi relativamente pequena, variando entre aproximadamente 2% e 5%, enquanto a redução no tempo de execução foi significativamente expressiva.

A Tabela 1 apresenta os resultados obtidos pelo algoritmo ACO utilizando as instâncias do PCV. As instâncias empregadas nos experimentos foram sugeridas pelo professor da disciplina por representarem diferentes níveis de complexidade e tamanhos de entrada, permitindo uma análise mais consistente da escalabilidade e eficiência do algoritmo ACO. Os testes foram realizados considerando execuções com e sem a utilização do mecanismo de radar, permitindo analisar o impacto dessa estratégia tanto no tempo de execução quanto na qualidade das soluções encontradas.

Tabela 1 – Resultados obtidos nos experimentos realizados com e sem a utilização do mecanismo de radar.

Instâncias	Nº de Cidades	Radar	Tempo (s)	Melhor Distância	Iterações
tsp29.tsp	29	Não	1.43	29811.00	60
tsp29.tsp	29	Não	1.43	29811.00	60
tsp38.tsp	38	Não	3.93	6856.00	73
tsp38.tsp	38	Sim	1.76	6974.00	73
tsp51.tsp	51	Não	10.34	455.00	91
tsp51.tsp	51	Sim	4.33	456.00	91
tsp100.tsp	100	Não	31.249	22483.00	57
tsp100.tsp	100	Sim	11.841	22472.00	57
tsp150.tsp	150	Não	127.88	6847.00	71
tsp150.tsp	150	Sim	40.642	6950.00	71
tsp436.tsp	436	Não	277.554	1754.00	50
tsp436.tsp	436	Sim	72.839	1746.00	50

Os experimentos realizados tiveram como objetivo avaliar o impacto da utilização do mecanismo de radar no desempenho do algoritmo de Otimização por Colônia de Formigas

(ACO). A Tabela 1 apresenta os resultados obtidos para diferentes instâncias do Problema do Caixeiro Viajante (TSP), considerando execuções com e sem a utilização do radar.

Os resultados demonstram que a principal contribuição do mecanismo proposto foi a redução significativa do tempo de execução em instâncias de maior porte. É importante destacar que, para instâncias com até 30 cidades, como a *tsp29*, o mecanismo de radar não é ativado, fazendo com que ambas as execuções utilizem o mesmo processo de busca. Dessa forma, as pequenas diferenças observadas nessa instância decorrem apenas de variações naturais da execução do algoritmo, não representando ganhos efetivos proporcionados pelo radar.

A partir das instâncias em que o mecanismo passa a atuar, observa-se uma redução progressiva no tempo de execução. Esse comportamento torna-se mais evidente à medida que o número de cidades aumenta. Na instância *tsp436*, por exemplo, o tempo de execução foi reduzido de 277,55 s para 72,84 s, correspondendo a uma diminuição superior a 73%.

Observa-se ainda que o ganho de desempenho cresce conforme o tamanho do problema aumenta. Isso ocorre porque o radar restringe o conjunto de cidades candidatas avaliadas pelas formigas, reduzindo o número de cálculos necessários para a tomada de decisão em cada etapa da construção das rotas. Dessa forma, o custo computacional da busca torna-se significativamente menor em instâncias de maior dimensão.

Em relação à qualidade das soluções encontradas, os resultados indicam que a utilização do radar não comprometeu de forma significativa o desempenho do algoritmo. Em algumas instâncias, como *tsp100* e *tsp436*, foram obtidas soluções ligeiramente melhores do que aquelas encontradas pela abordagem convencional. Já nas demais instâncias, as diferenças observadas permaneceram pequenas, indicando que a redução do espaço de busca não afetou substancialmente a capacidade de exploração do algoritmo.

Outro aspecto relevante é que o número de iterações permaneceu inalterado entre as execuções com e sem radar para todas as instâncias avaliadas. Isso evidencia que a redução do tempo de execução não foi resultado de uma diminuição no esforço de busca, mas sim de uma otimização no processo de seleção das cidades candidatas durante a construção das soluções.

De maneira geral, os resultados obtidos indicam que o mecanismo de radar constitui uma estratégia eficiente para acelerar a execução do algoritmo ACO em instâncias de médio e grande porte. Além de proporcionar reduções expressivas no tempo computacional, a abordagem manteve a qualidade das soluções encontradas, demonstrando um equilíbrio favorável entre eficiência e precisão.

As Tabelas 2 e 3 apresentam a evolução dos níveis de feromônio ao longo das iterações do algoritmo ACO para a instância analisada. A Tabela 2 corresponde à execução realizada sem a utilização do mecanismo de radar, enquanto a Tabela 3 apresenta os resultados obtidos com a aplicação desse mecanismo.

Tabela 2 – Evolução dos feromônios ao longo das iterações do algoritmo ACO **sem radar** na instância **tsp51.tsp**. Fonte: Elaborado pelos autores.

Rodada	Intensidade Máxima	Aresta Mais Forte	Média Global	Conhecimento Total
1	19.27	AT ↔ AY	1.6204	4214.6
11	155.58	AT ↔ AY	6.4025	16652.9
21	208.04	AT ↔ AY	8.2817	21540.6
31	231.21	S ↔ AO	9.1250	23734.1
41	241.41	AT ↔ AY	9.4981	24704.6
51	248.90	AT ↔ AY	9.7715	25415.8
61	252.57	AT ↔ AY	9.9085	25771.9
91	256.03	AT ↔ AY	10.0404	26115.1

Tabela 3 – Evolução dos feromônios ao longo das iterações do algoritmo ACO **com radar** na instância **tsp51.tsp**. Fonte: Elaborado pelos autores.

Rodada	Intensidade Máxima	Aresta Mais Forte	Média Global	Conhecimento Total
1	20.78	AT ↔ AY	1.7133	4456.3
11	159.37	AT ↔ AY	6.5826	17121.3
21	213.38	AT ↔ AY	8.4839	22066.6
31	234.52	AT ↔ AY	9.2372	24025.8
41	242.33	AT ↔ AY	9.5171	24754.0
51	247.45	AT ↔ AY	9.7087	25252.4
61	249.95	AT ↔ AY	9.8035	25499.0
91	253.49	S ↔ AO	9.9410	25856.5

As métricas apresentadas nas Tabelas 2 e 3 foram utilizadas para acompanhar a evolução do processo de aprendizado coletivo das formigas ao longo das iterações do algoritmo ACO para a instância **tsp51.tsp**, considerando execuções sem e com a utilização do mecanismo de radar.. A seguir, são descritos os indicadores analisados:

- **Rodada** Representa a iteração atual do algoritmo. Em cada rodada, todas as formigas constroem novas soluções para o problema e, posteriormente, os níveis de feromônio são atualizados de acordo com a qualidade das rotas encontradas.
- **Intensidade Máxima** Corresponde ao maior valor de feromônio observado em uma aresta da rede durante determinada iteração. Esse indicador permite identificar o nível máximo de reforço aplicado pela colônia a um caminho específico, refletindo o grau de preferência coletiva por essa conexão.

- **Aresta Mais Forte** Identifica a aresta que possui a maior concentração de feromônio na rodada analisada. Em geral, essa conexão está associada às rotas mais promissoras encontradas pelas formigas e tende a aparecer com frequência nas melhores soluções obtidas pelo algoritmo.
- **Média Global** Representa a média dos níveis de feromônio considerando todas as arestas do grafo. Essa métrica fornece uma visão geral da distribuição do conhecimento acumulado pela colônia ao longo do processo de busca.
- **Conhecimento Total** Corresponde à soma de todos os valores presentes na matriz de feromônios. Esse indicador expressa a quantidade total de informação compartilhada pelas formigas durante a execução do algoritmo, permitindo avaliar a evolução do aprendizado coletivo da colônia.

Observa-se que ambas as abordagens apresentam um comportamento de convergência bastante semelhante. Em ambos os cenários, a intensidade máxima de feromônio cresce rapidamente nas primeiras iterações, passando de aproximadamente 20 unidades na primeira rodada para valores superiores a 200 unidades por volta da vigésima iteração. Após esse período, o crescimento torna-se gradualmente menor, indicando a estabilização do processo de aprendizado coletivo das formigas e a convergência para rotas mais promissoras.

A análise das arestas com maior concentração de feromônio mostra que a conexão $AT \leftrightarrow AY$ foi predominante durante a maior parte da execução. Na abordagem sem radar, houve uma breve mudança na rodada 31, quando a aresta $S \leftrightarrow AO$ apresentou a maior intensidade de feromônio, mas a liderança retornou posteriormente para $AT \leftrightarrow AY$. Já na execução com radar, a aresta $AT \leftrightarrow AY$ permaneceu como a mais forte durante quase toda a execução, sendo superada por $S \leftrightarrow AO$ apenas nas rodadas finais. Esse comportamento indica que ambas as abordagens identificaram estruturas semelhantes na construção das melhores rotas.

A média global de feromônios também apresentou crescimento contínuo nas duas configurações. Na execução sem radar, esse indicador evoluiu de 1,6204 para 10,0404, enquanto na execução com radar passou de 1,7133 para 9,9410. Embora existam pequenas diferenças entre os valores finais, o comportamento geral das curvas permanece bastante próximo, evidenciando que o mecanismo de radar não altera significativamente a distribuição global dos feromônios na rede.

Resultado semelhante pode ser observado no indicador de conhecimento total acumulado. Na primeira rodada, a versão com radar apresentou um valor ligeiramente superior ao

obtido sem radar (4456,3 contra 4214,6), sugerindo um reforço inicial mais intenso das rotas exploradas. Ao final da execução, os valores alcançaram 26115,1 na abordagem sem radar e 25856,5 na abordagem com radar, demonstrando que ambas as execuções acumularam níveis de conhecimento bastante próximos ao longo do processo de otimização.

Comparando-se os valores finais, verifica-se que a intensidade máxima dos feromônios atingiu 256,03 na versão sem radar e 253,49 na versão com radar. Da mesma forma, os indicadores de média global e conhecimento total apresentaram diferenças relativamente pequenas quando comparados à magnitude dos valores observados durante a execução. Esses resultados indicam que o mecanismo de radar preserva as principais características do processo de aprendizado do ACO, mantendo um padrão de reforço muito semelhante ao obtido pela abordagem convencional.

Dessa forma, conclui-se que a utilização do radar reduz o esforço computacional necessário durante a construção das rotas sem comprometer significativamente o mecanismo de comunicação indireta entre as formigas, conhecido como estigmergia. A elevada similaridade entre a evolução dos feromônios nos dois cenários sugere que o método proposto mantém a capacidade de convergência do algoritmo para soluções de alta qualidade, ao mesmo tempo em que contribui para a melhoria da eficiência computacional.

As Figuras 2, 3, 4 e 5 apresentam visualizações geradas por meio da biblioteca Matplotlib, da linguagem Python, ao final dos experimentos realizados sobre as instâncias analisadas. Essas representações gráficas têm como objetivo facilitar a compreensão do funcionamento do algoritmo e auxiliar na interpretação das operações realizadas durante sua execução.

A Figura 2 ilustra a distribuição espacial das cidades com base nas coordenadas fornecidas pelo arquivo da instância utilizada. A Figura 3 apresenta a melhor rota encontrada pelo algoritmo ACO sem a utilização do mecanismo de radar, enquanto a Figura 4 mostra a melhor rota obtida com a aplicação desse mecanismo.

Por fim, a Figura 5 apresenta a evolução da melhor solução encontrada ao longo das iterações do algoritmo, além de comparar o resultado obtido pelo ACO com soluções produzidas por diferentes estratégias gulosas. Essa comparação permite avaliar tanto a capacidade de convergência do algoritmo quanto a qualidade das rotas encontradas em relação a abordagens mais simples de construção de soluções.

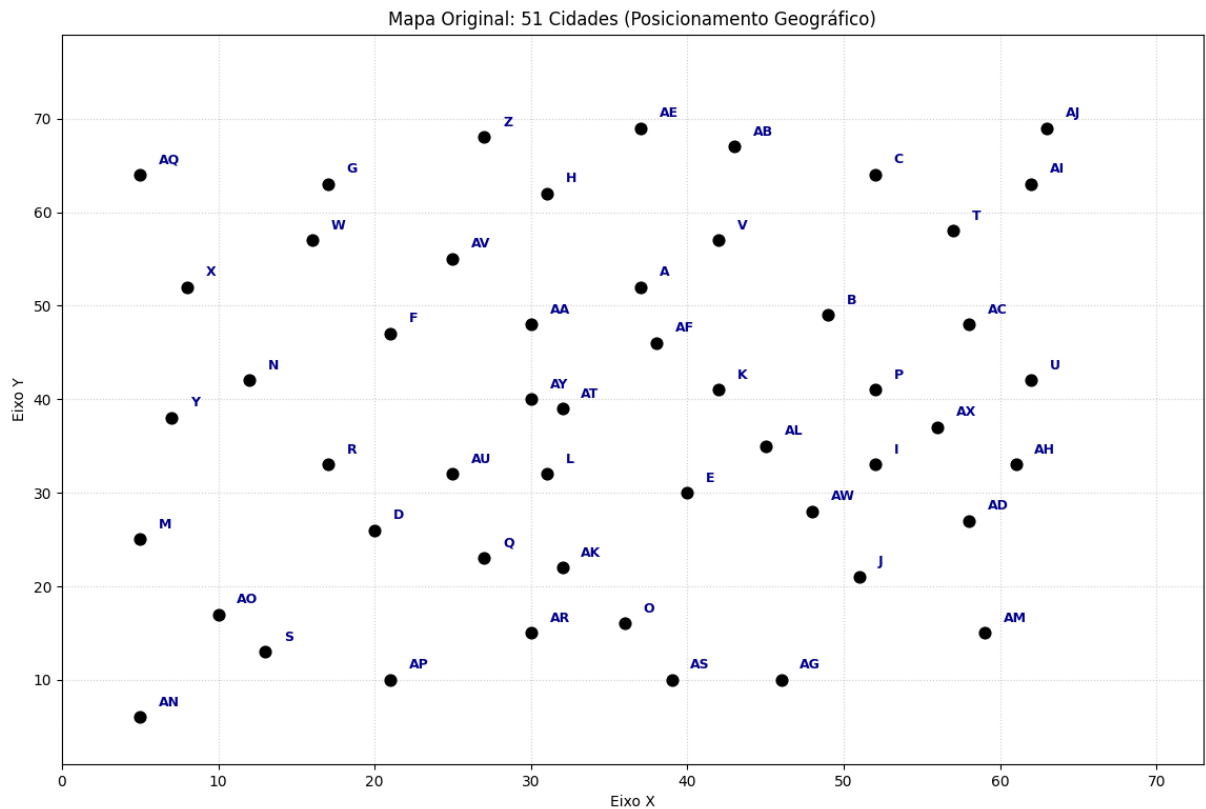


Figura 2 – Distribuição espacial das 51 cidades utilizadas no experimento da instância **tsp51.tsp**. Os pontos representam as cidades do problema TSP, servindo como base para a construção das rotas pelo algoritmo ACO. Fonte: Elaborado pelos autores.

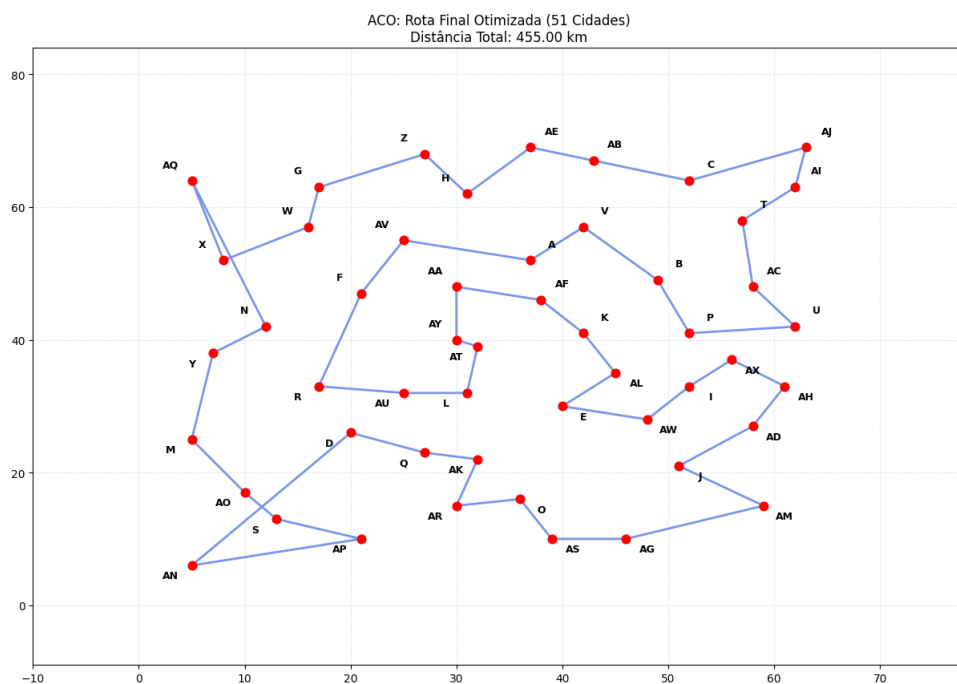


Figura 3 – Melhor rota encontrada pelo algoritmo ACO sem a utilização do mecanismo de radar para a instância com 51 cidades. A rota apresentada corresponde à menor distância obtida ao final do processo de otimização. Fonte: Elaborado pelos autores.

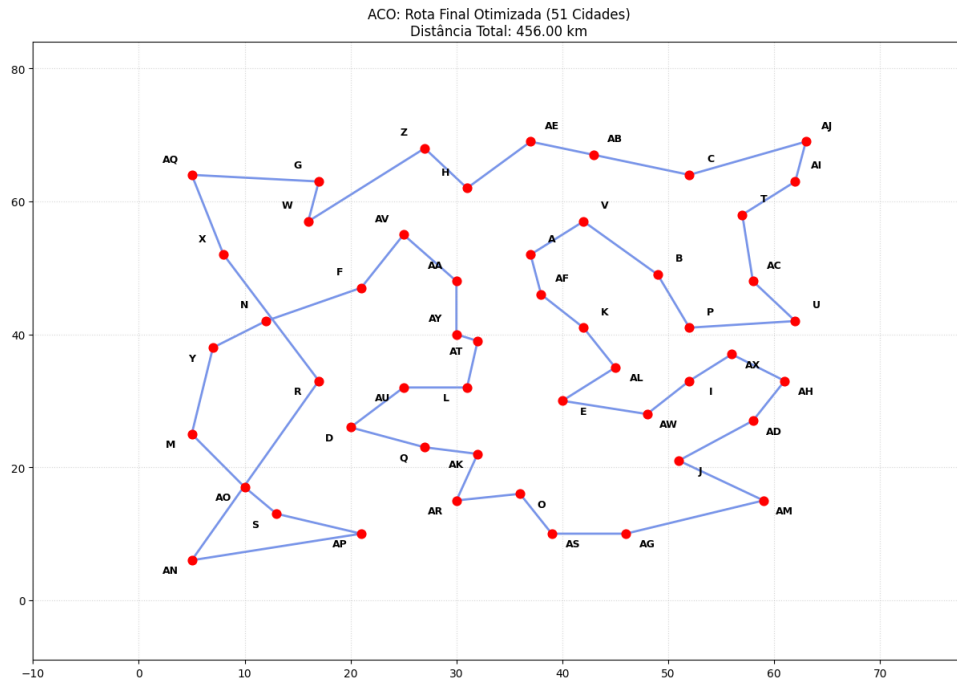


Figura 4 – Melhor rota encontrada pelo algoritmo ACO com a utilização do mecanismo de radar para a instância com 51 cidades. Observa-se que a estratégia de restrição de vizinhança preserva a qualidade da solução obtida. Fonte: Elaborado pelos autores.

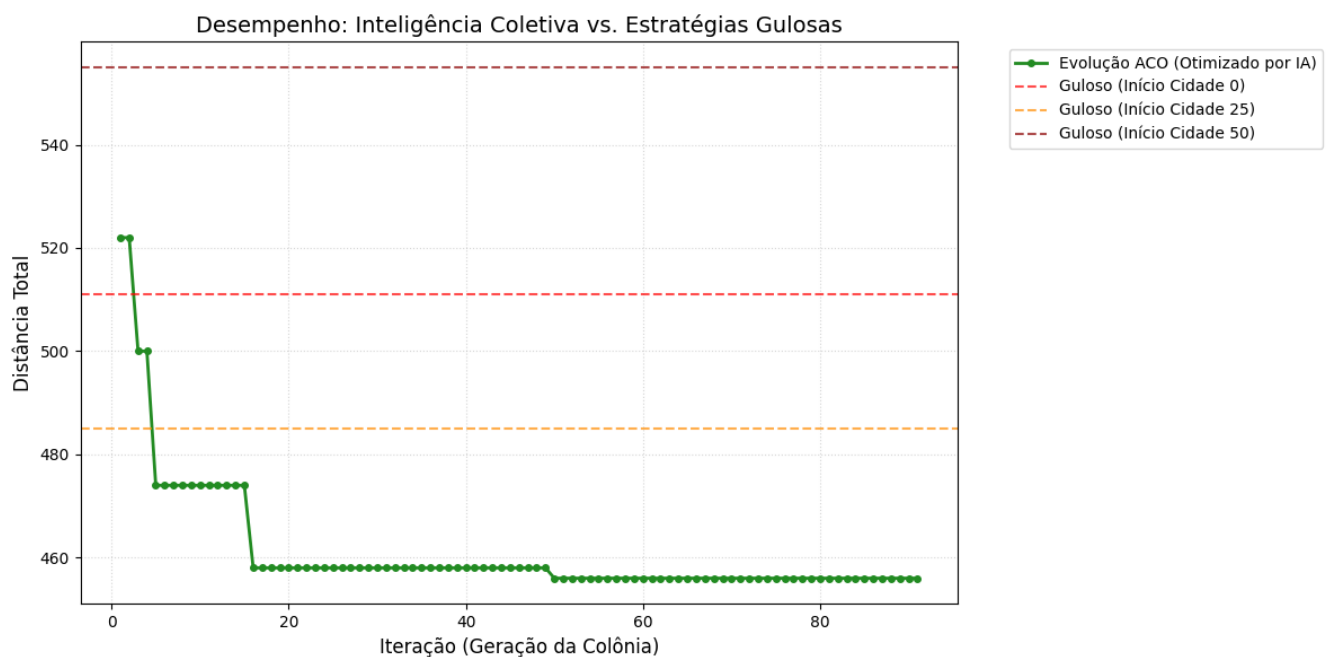


Figura 5 – Comparação do desempenho do algoritmo ACO na instância com 51 cidades, evidenciando a redução do tempo de execução proporcionada pela utilização do mecanismo de radar ao longo do processo de otimização. Fonte: Elaborado pelos autores.

A Figura 5 apresenta a evolução da melhor solução encontrada pelo algoritmo ACO ao longo das iterações, bem como os resultados obtidos por diferentes estratégias gulosas aplicadas à instância `tsp51.tsp`.

Observa-se que o algoritmo ACO inicia sua execução com soluções em torno de 522 unidades de distância total e apresenta uma rápida melhoria nas primeiras iterações. Nas gerações iniciais ocorre uma redução significativa no comprimento da rota, passando de aproximadamente 522 para valores próximos de 474 unidades. Esse comportamento evidencia a capacidade da colônia de explorar diferentes caminhos e reforçar progressivamente as rotas mais promissoras por meio do mecanismo de atualização dos feromônios.

Ao longo das iterações seguintes, novas melhorias são encontradas, reduzindo a distância para cerca de 458 unidades por volta da décima sexta geração. Posteriormente, por volta da quinquagésima iteração, o algoritmo encontra sua melhor solução, com distância aproximada de 456 unidades, permanecendo estável até o final da execução. Esse comportamento indica que a colônia convergiu para uma rota de alta qualidade, não encontrando alternativas significativamente melhores nas iterações subsequentes.

Em contraste, as estratégias gulosas apresentaram resultados inferiores. Dependendo da cidade inicial escolhida, as distâncias obtidas foram aproximadamente 485, 511 e 556 unidades. Essa variação evidencia uma característica importante dos métodos gulosos: a forte dependência do ponto de partida e das decisões locais realizadas durante a construção da rota.

Comparando-se os melhores resultados, verifica-se que o ACO encontrou uma solução cerca de 6% menor que a melhor estratégia gulosa avaliada (485 unidades) e aproximadamente 18% menor que a pior solução gulosa observada (556 unidades). Esses resultados demonstram a eficácia do mecanismo de inteligência coletiva empregado pelo algoritmo, que consegue explorar melhor o espaço de busca e evitar soluções localmente atrativas, mas globalmente subótimas.

Outro aspecto relevante observado no gráfico é a estabilidade da convergência. Após encontrar uma solução de elevada qualidade, o algoritmo mantém praticamente o mesmo valor durante as iterações restantes, indicando que o processo de reforço dos feromônios foi capaz de consolidar os melhores caminhos identificados pela colônia.

Dessa forma, os resultados indicam que o algoritmo ACO supera as estratégias gulosas avaliadas, produzindo rotas mais curtas e de melhor qualidade. Isso reforça a vantagem dos mecanismos de busca cooperativa e aprendizado coletivo em problemas combinatórios complexos, como o Problema do Caixeiro Viajante.

3.4 PONTOS FORTES

A abordagem proposta apresentou resultados satisfatórios tanto em termos de desempenho computacional quanto na qualidade das soluções geradas. A introdução do mecanismo de radar permitiu reduzir significativamente o número de cidades avaliadas durante a construção das rotas, diminuindo o tempo de execução do algoritmo sem comprometer seu processo de aprendizado baseado em feromônios. Os experimentos realizados mostraram que a estratégia preservou a capacidade de convergência do ACO, mantendo resultados competitivos mesmo em instâncias de maior porte.

Além dos resultados diretamente relacionados ao objetivo principal do trabalho, as implementações complementares desenvolvidas para o ACO contribuíram significativamente para a compreensão do funcionamento dessas abordagens. Em particular, os recursos de visualização implementados possibilitaram a análise gráfica do comportamento do algoritmo ao longo de sua execução, facilitando a interpretação dos resultados, a validação das soluções encontradas e o entendimento do processo de otimização e convergência.

3.5 LIMITAÇÕES

A implementação desenvolvida apresenta algumas limitações decorrentes das decisões adotadas durante sua execução. Embora tais escolhas tenham contribuído para a viabilidade da abordagem proposta, elas também introduzem restrições que podem influenciar o desempenho do algoritmo em determinados cenários.

- **Custo de Inicialização:** A leitura dos arquivos de configuração e o carregamento da biblioteca numpy adicionam um custo computacional ao início da execução do programa. Em ambientes sujeitos a limites rigorosos de tempo, esse custo reduz a parcela do tempo disponível para a execução efetiva das iterações do algoritmo.
- **Restrição do Espaço de Busca:** O mecanismo de *radar* foi adotado com o objetivo de reduzir a quantidade de candidatos avaliados durante a construção das soluções. Embora essa estratégia contribua para diminuir o esforço computacional, ela também restringe o conjunto de alternativas exploradas pelas formigas, podendo limitar a identificação de rotas potencialmente mais vantajosas em algumas instâncias.
- **Generalização dos Parâmetros Inferidos:** Os valores de α e β são definidos a partir de informações previamente obtidas pelo modelo de Inteligência Artificial. Dessa forma, a

qualidade dessa parametrização depende da representatividade das instâncias utilizadas durante a etapa de treinamento. Em situações nas quais as características da instância analisada diferem significativamente dos dados utilizados na construção do modelo, o desempenho da inferência pode ser reduzido.

3.6 POSSÍVEIS MELHORIAS

Com base nos resultados obtidos e nas limitações identificadas durante os experimentos, algumas melhorias podem ser consideradas em trabalhos futuros com o objetivo de ampliar a qualidade das soluções e a eficiência computacional da abordagem proposta.

- **Integração com Algoritmos de Busca Local:** Uma possível extensão consiste na aplicação de heurísticas de refinamento local, sobre as melhores soluções geradas pelo algoritmo ACO. Essa estratégia pode contribuir para a remoção de arestas desnecessárias e para o refinamento das rotas encontradas, possibilitando a obtenção de soluções de menor custo.
- **Redução do Custo Computacional:** Outra possibilidade é a revisão das dependências utilizadas durante a execução do algoritmo, buscando reduzir o custo associado à inicialização do programa. Além disso, trechos da implementação podem ser reescritos utilizando ferramentas de otimização ou linguagens compiladas, com o objetivo de aumentar o número de iterações executadas dentro do tempo disponível.
- **Adaptação Dinâmica de Parâmetros:** O modelo atual utiliza valores inferidos para os parâmetros de controle antes do início da execução. Como trabalho futuro, pode-se investigar mecanismos capazes de ajustar parâmetros como a taxa de evaporação do feromônio (ρ) ao longo da busca. Essa adaptação permitiria modificar o comportamento do algoritmo em resposta ao progresso obtido, favorecendo o equilíbrio entre exploração e intensificação.

4 CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a avaliação de uma abordagem para o Problema do Caixeiro Viajante (PCV) baseada na metaheurística de Otimização por Colônia de Formigas (ACO) associada à utilização de parâmetros inferidos por um modelo de Inteligência Artificial. A proposta teve como objetivo combinar a capacidade exploratória do ACO com uma parametrização adaptada às características das instâncias analisadas.

Os resultados obtidos indicaram que a utilização do mecanismo de *radar* contribuiu para a redução do esforço computacional durante a construção das soluções, tornando viável a execução do algoritmo em instâncias de diferentes dimensões. Embora a restrição do espaço de busca possa limitar a exploração de determinadas alternativas, a estratégia permitiu reduzir a quantidade de candidatos avaliados em cada etapa da busca, favorecendo o desempenho computacional da abordagem.

Também foi observado que a utilização de parâmetros inferidos previamente possibilitou a obtenção de soluções competitivas desde as primeiras iterações do algoritmo. Por outro lado, a etapa de inferência e o carregamento das bibliotecas necessárias introduziram um custo computacional adicional, reduzindo a quantidade de iterações disponíveis em cenários com restrições rígidas de tempo.

De modo geral, os experimentos demonstraram que a combinação entre ACO e Inteligência Artificial constitui uma alternativa viável para o tratamento do PCV, apresentando resultados satisfatórios nas instâncias avaliadas. Além disso, a análise das limitações identificadas e das possíveis melhorias propostas fornece subsídios para trabalhos futuros voltados ao aperfeiçoamento da qualidade das soluções e da eficiência computacional da abordagem desenvolvida.

5 USO DE FERRAMENTAS DE IA

Nesta seção são apresentadas as ferramentas de Inteligência Artificial utilizadas durante o desenvolvimento deste trabalho, bem como suas respectivas finalidades. O uso dessas ferramentas teve como objetivo auxiliar na organização da documentação, revisão textual, esclarecimento de conceitos teóricos e apoio na elaboração de elementos complementares do relatório, sem substituir a análise, implementação e validação realizadas pelos autores.

5.1 FERRAMENTAS UTILIZADAS

As ferramentas utilizadas para a elaboração deste trabalho e do presente relatório foram:

- ChatGPT
- Consensus
- Gemini

5.2 FINALIDADES DE USO

O resumo das ferramentas de Inteligência Artificial utilizadas durante o desenvolvimento deste trabalho e Relatório é apresentado na Tabela 4.

Tabela 4 – Ferramentas de Inteligência Artificial utilizadas durante o desenvolvimento do trabalho.

Ferramenta	Finalidade	Utilização no Trabalho	Categoria
ChatGPT	Assistente de IA generativa	Auxílio na revisão textual, organização da documentação, explicação de conceitos relacionados ao Problema do Caixeiro Viajante e ao algoritmo ACO, além de suporte na elaboração de pseudocódigos e formatação em $\text{\LaTeX}$.	Revisão de ortografia e gramática, Correção e formatação em $\text{\LaTeX}$
Consensus	Pesquisa científica baseada em artigos acadêmicos	Consulta de literatura científica para obtenção de conceitos, definições e evidências relacionadas ao Problema do Caixeiro Viajante, algoritmos de otimização e metaheurísticas, contribuindo para a fundamentação teórica do trabalho.	Pesquisa Científica
Gemini	Pesquisa e apoio conceitual	Consulta complementar para validação de conceitos, comparação de abordagens e obtenção de explicações alternativas sobre os temas estudados.	Refatoração do código

5.3 PROMPTS UTILIZADOS

A Tabela 5 apresenta exemplos dos principais prompts utilizados durante a interação com as ferramentas de Inteligência Artificial empregadas neste trabalho.

Tabela 5 – Principais prompts utilizados durante o desenvolvimento do trabalho.

Finalidade	Prompt Utilizado
Revisão textual	"Revise este texto para um relatório acadêmico, corrigindo erros gramaticais e melhorando a clareza e a formalidade da escrita."
Explicação conceitual	"Explique o funcionamento do algoritmo Ant Colony Optimization (ACO)"
Formatação em LaTeX	"Transforme esta equação matemática em código LaTeX e organize sua apresentação para um relatório de trabalho acadêmico."
Pseudocódigo	"Em Latex, Gere um pseudocódigo, em formato acadêmico, a partir deste trecho de código Python."
Pesquisa bibliográfica	"Apresente referências e conceitos relacionados ao Problema do Caixeiro Viajante e às metaheurísticas inspiradas em colônias de formigas."
Relatório	"Melhore a redação deste parágrafo para que fique adequada a um relatório técnico de graduação."
Refatorar o Código	"me auxilie na refatoração deste código baseado no algoritmo ACO"
Análise do Algoritmo	"Analisando a proposta do trabalho, identifique e explique os pontos fortes do algoritmo proposto, bem como suas limitações decorrente da integração do ACO com a inferência de hiperparâmetros via aprendizado de máquina."
Geração/Melhoria da Figura 1 da seção 2.1	"Elabore uma figura didática baseada na descrição conceitual do algoritmo de Colônia de Formigas (ACO), representando o processo de exploração, deposição de feromônios e convergência para a rota ótima, tomando como referência a imagem fornecida."

Este último prompt foi utilizado exclusivamente para a elaboração de uma figura ilustrativa destinada a complementar a explicação do algoritmo ACO. A imagem gerada teve como objetivo facilitar a compreensão do processo de exploração, deposição de feromônios e convergência para a rota de menor custo, tomando como base a explicação apresentada por (Ponto Ótimo, 2017). Ressalta-se que a figura possui caráter didático e não representa resultados experimentais obtidos neste trabalho.

REFERÊNCIAS

AKHTAR, A. Evolution of ant colony optimization algorithm - a brief literature review. **ArXiv**, abs/1908.08007, 2019.

Ponto Ótimo. **Algoritmo Colônia de Formigas - Aplicado Ao Problema do Caixeiro Viajante**. 2017. <<https://youtu.be/sDBe6R0axAY?si=Nft4GaV3XkddBDhF&t=193>>.

SACRAMENTO, D.; PISINGER, D.; RØPKE, S. An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones. **Transportation Research Part C: Emerging Technologies**, 2019.

WANG, J.; XIAO, C.; WANG, S.; RUAN, Y. Reinforcement learning for the traveling salesman problem: Performance comparison of three algorithms. **The Journal of Engineering**, 2023.